GPUMODE Lecture Study Notes: Lecture 64 MultiGPU Programming

Core Problem the Lecture Addresses

The lecture explains how to move from a single-GPU program to a multiGPU program while preserving correctness, performance, and a clear mental model of communication. The concrete example is a Jacobi solver for the two-dimensional Laplace equation, but the deeper topic is how to structure a GPU application when the problem is too large, too slow, or too distributed for one GPU.

The central technical problem is:

How do we split a computation across GPUs while minimizing communication overhead?

A single GPU program mostly worries about thread parallelism, memory coalescing, occupancy, shared memory, registers, and kernel efficiency. A multiGPU program adds another level of performance engineering:

- The global problem must be decomposed across processes or devices.
- Neighboring GPUs often need boundary data from each other.
- Communication may go through NVLink, PCIe, InfiniBand, host memory, or network interface cards.
- Synchronization can easily destroy scaling if it lies on the critical path.
- Communication libraries such as MPI, NCCL, and NVSHMEM provide different programming models and different opportunities for overlap.

The lecture frames multiGPU programming around two scaling goals.

Strong Scaling

In strong scaling, the global problem size is fixed, and more GPUs are used to solve the same problem faster. If the single-GPU runtime is T_1 and the runtime on p GPUs is T_p, the ideal runtime is:

$$T_p^{\text{ideal}} = \frac{T_1}{p}.$$

The parallel efficiency is:

$$E_p = \frac{T_1}{pT_p}.$$

If $E_p = 1$, scaling is perfect. In practice, $E_p < 1$ because of communication, synchronization, load imbalance, launch overhead, and nonparallel portions of the program.

Strong scaling is hard because the local work per GPU decreases as p grows. If communication does not shrink proportionally, communication becomes a larger fraction of the runtime.

Weak Scaling

In weak scaling, the amount of work per GPU is held approximately constant while the total problem size grows with the number of GPUs. Ideally:

$$T_p \approx T_1.$$

Weak scaling is useful when the real goal is to solve a larger problem, not necessarily to solve the same problem faster. It is common in climate simulation, molecular dynamics, lattice QCD, large-scale training, and large batch inference.

The Main Trade-Off

The central trade-off is:

$$\text{More GPUs} \Rightarrow \text{more parallel compute capacity}$$

but also:

$$\text{More GPUs} \Rightarrow \text{more communication and synchronization.}$$

The engineering goal is to make communication small, fast, and hidden behind useful computation.

Required Background

GPU Execution Model

A CUDA program launches kernels onto a GPU. A kernel is executed by a grid of thread blocks. Each block contains threads. Threads are executed in groups of 32 called warps. Blocks are scheduled onto streaming multiprocessors, usually abbreviated as SMs.

Important GPU concepts for this lecture are:

- **Thread**: the smallest CUDA execution unit visible to the programmer.
- **Warp**: a group of 32 threads that execute in SIMD-style lockstep.
- **Thread block**: a group of threads that can cooperate through shared memory and synchronization.
- **Grid**: the full collection of thread blocks launched by a kernel.
- **SM**: the hardware unit that schedules warps and executes instructions.
- **Global memory**: large device memory visible to all threads, with high bandwidth but high latency.
- **Shared memory**: programmer-managed on-chip memory local to an SM.
- **Registers**: per-thread fast storage.
- **CUDA stream**: an ordered queue of asynchronous GPU operations.

A CUDA stream gives ordering guarantees. If operations A, B, and C are enqueued into the same stream, then:

$$A \prec B \prec C.$$

This means B cannot begin before A is complete, and C cannot begin before B is complete. Different streams can execute concurrently if resources permit.

MultiGPU Process Model

The lecture emphasizes a common production pattern:

$$\text{one process} \leftrightarrow \text{one GPU}.$$

If a node has 8 GPUs, a typical distributed job launches 8 processes on that node. Each process is assigned one GPU and is given a unique rank. This model is common in MPI, NCCL, PyTorch Distributed, JAX distributed setups, and many HPC applications.

Reasons this model is preferred:

- Each process owns one CUDA context.
- Device selection is simple.
- The rank-to-GPU mapping is explicit.
- Communication libraries can reason about topology more easily.
- It avoids multiple processes oversubscribing the same GPU.
- It avoids one process manually switching between several devices.

It is possible to use multiple GPUs from one process, but the one-process-per-GPU model is usually simpler and faster for distributed workloads.

MPI Concepts

MPI stands for Message Passing Interface. It is a standard for communication between processes.

The key MPI concepts are:

- **Rank**: the unique integer ID of a process inside a communicator.
- **Size**: the number of participating processes.
- **Communicator**: a group of ranks that can communicate.
- **Point-to-point communication**: operations such as send and receive.
- **Collective communication**: operations such as broadcast, reduce, all-reduce, scatter, and gather.

The default communicator is `MPI_COMM_WORLD`. If there are p total processes, ranks are usually numbered:

$$0, 1, 2, \ldots, p-1.$$

NCCL Concepts

NCCL stands for NVIDIA Collective Communication Library. It began as a high-performance GPU collective communication library, especially for deep learning all-reduce. It now also supports point-to-point communication.

NCCL is important because:

- It is optimized for GPU buffers.
- It understands GPU topology.
- It can use NVLink, PCIe, and network paths efficiently.
- NCCL operations are stream ordered.

- Communication can be naturally ordered with CUDA kernels in the same stream.

The stream-ordered interface is the key advantage for this lecture.

NVSHMEM Concepts

NVSHMEM is NVIDIA's GPU-oriented implementation of a partitioned global address space programming model. It follows ideas from OpenSHMEM.

Instead of requiring both sender and receiver to participate, NVSHMEM supports one-sided communication. One GPU can write data into another GPU's symmetric memory region.

NVSHMEM introduces the term processing element, abbreviated PE. A PE is analogous to a rank.

The important idea is the **symmetric heap**. Allocations made with `nvshmem_malloc` are visible in a symmetric way across PEs. This allows one PE to compute the address of corresponding objects on another PE.

Main Concepts

The Example Problem: Two-Dimensional Laplace Equation

The lecture uses a Jacobi solver for the two-dimensional Laplace equation:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0.$$

The unknown function $u(x, y)$ is solved on a two-dimensional grid.

Boundary conditions are:

- Dirichlet boundary conditions on the left and right boundaries.
- Periodic boundary conditions on the top and bottom boundaries.

The periodic top-bottom boundary means the two-dimensional domain behaves like the surface of a cylinder. Moving above the top row wraps to the bottom row, and moving below the bottom row wraps to the top row.

Jacobi Iteration

The Jacobi update computes each new grid value as the average of its four nearest neighbors:

$$u_{\text{new}}(i, j) = \frac{1}{4}\left(u(i-1, j) + u(i+1, j) + u(i, j-1) + u(i, j+1)\right).$$

Here:

- i is the row index.
- j is the column index.
- u_{new} is the next iterate.
- u is the previous iterate.

The iteration continues until convergence. A common convergence metric is the L_2 norm of the difference between the old and new solution:

$$\|u_{\text{new}} - u\|_2 = \sqrt{\sum_{i,j} \left(u_{\text{new}}(i,j) - u(i,j)\right)^2}.$$

In distributed form, each GPU computes a local sum of squared differences:

$$s_r = \sum_{(i,j)\in\Omega_r} \left(u_{\text{new}}(i,j) - u(i,j)\right)^2,$$

where Ω_r is the subdomain owned by rank r. The global norm is:

$$\|u_{\text{new}} - u\|_2 = \sqrt{\sum_{r=0}^{p-1} s_r}.$$

The global sum is computed with an all-reduce.

Linearized Two-Dimensional Indexing

The solver stores a two-dimensional grid in a one-dimensional C-style row-major array. If the number of columns is n_x, the linear index for row i_y and column i_x is:

$$\text{idx}(i_y, i_x) = i_y n_x + i_x.$$

Neighbor indices are:

$$\begin{aligned}
\text{left} &= i_y n_x + (i_x - 1),\\
\text{right} &= i_y n_x + (i_x + 1),\\
\text{up} &= (i_y - 1) n_x + i_x,\\
\text{down} &= (i_y + 1) n_x + i_x.
\end{aligned}$$

This indexing matters because horizontal rows are contiguous in memory. Therefore, decomposing the domain into horizontal stripes makes boundary rows contiguous, which simplifies communication.

Domain Decomposition

To use multiple GPUs, the global domain is split into smaller subdomains. The lecture discusses several possible decompositions:

- Horizontal stripes.
- Vertical stripes.
- Two-dimensional tiles.

The lecture chooses horizontal stripes because each row is contiguous in memory. If each GPU owns a set of rows, then exchanging a boundary row requires sending one contiguous buffer of length n_x.

If the global domain has n_y rows and p GPUs, a simple balanced decomposition gives each rank approximately:

$$n_y^{\text{local}} \approx \frac{n_y}{p}$$

rows, plus halo rows.

Halo Exchange

Once the domain is decomposed, each GPU needs values from neighboring GPUs. For the Jacobi stencil, each grid point depends on its top and bottom neighbors. Therefore, when the domain is split horizontally, each rank must exchange boundary rows with its top and bottom neighbors.

A halo row is a ghost row stored locally but computed by a neighboring rank.

For rank r, define:

$$r_{\text{top}} = \begin{cases} r-1, & r > 0, \\ p-1, & r = 0, \end{cases}$$

$$r_{\text{bottom}} = \begin{cases} r+1, & r < p-1, \\ 0, & r = p-1. \end{cases}$$

This implements periodic wraparound.

The halo exchange can be summarized as:

send first interior row to top neighbor,
receive bottom halo row from bottom neighbor,
send last interior row to bottom neighbor,
receive top halo row from top neighbor.

Visual Concept

Draw a tall rectangle split horizontally into several colored stripes. Each stripe is one GPU's local domain. Add one ghost row above and below each stripe. Draw arrows showing the bottom interior row of one GPU copied into the top halo row of its lower neighbor, and similarly in the opposite direction.

Hardware-Level Explanation

Single-GPU Execution

On a single GPU, the Jacobi update is a memory-intensive stencil operation. Each point reads four neighboring values and writes one output value. Ignoring cache reuse, each update uses roughly:

$$4 \text{ reads} + 1 \text{ write}.$$

For 32-bit floats, the memory traffic per point is approximately:

$$5 \times 4 = 20 \text{ bytes}.$$

The floating-point work per point is approximately:

$$3 \text{ additions} + 1 \text{ multiplication}$$

or roughly 4 floating-point operations. The arithmetic intensity is:

$$I = \frac{\text{FLOPs}}{\text{bytes}} \approx \frac{4}{20} = 0.2 \text{ FLOPs/byte}.$$

This is low, so the kernel is usually memory-bandwidth-bound rather than compute-bound.

MultiGPU Communication Paths

When one GPU sends data to another GPU, the path depends on the system topology and software stack.

GPU Direct Peer-to-Peer

If GPUs are on the same node and connected by NVLink, GPU Direct peer-to-peer can move data directly between GPU memories:

$$\text{GPU}_0 \rightarrow \text{NVLink} \rightarrow \text{GPU}_1.$$

This avoids staging through CPU memory.

GPU Direct RDMA

If GPUs are on different nodes and the network supports GPU Direct RDMA, the network interface card can directly read from or write to GPU memory:

$$\text{GPU memory} \rightarrow \text{NIC} \rightarrow \text{network} \rightarrow \text{NIC} \rightarrow \text{GPU memory}.$$

RDMA stands for remote direct memory access. It reduces latency and avoids extra copies through host memory.

CUDA-Aware MPI Without GPU Direct RDMA

If MPI understands CUDA buffers but cannot use GPU Direct RDMA, it may internally stage through pinned host memory:

$$\text{GPU} \rightarrow \text{pinned host buffer} \rightarrow \text{network} \rightarrow \text{pinned host buffer} \rightarrow \text{GPU}.$$

The user code can remain unchanged, but performance is lower.

Non-CUDA-Aware MPI

If MPI does not understand CUDA device pointers, the programmer must manually copy data to the host before calling MPI:

$$\text{cudaMemcpyDeviceToHost} \rightarrow \text{MPI\_Send} \rightarrow \text{MPI\_Recv} \rightarrow \text{cudaMemcpyHostToDevice}.$$

This introduces extra copies and extra synchronization.

Unified Virtual Addressing

CUDA unified virtual addressing allows CPU and GPU memory allocations to exist in one virtual address space. A pointer value can be inspected to determine whether it points to host memory or device memory.

Communication libraries can use pointer attributes to decide whether a buffer is a host buffer or a GPU buffer. Conceptually:

$$\text{pointer} \rightarrow \text{query attributes} \rightarrow \text{choose host path or GPU path.}$$

This is why CUDA-aware MPI can accept GPU buffers directly.

Critical Path

The critical path is the sequence of operations that determines total runtime. If communication lies on the critical path, it directly increases runtime. If communication is overlapped with computation, its cost can be hidden.

A simplified iteration without overlap is:

$$T_{\text{iter}} = T_{\text{compute}} + T_{\text{comm}} + T_{\text{sync}} + T_{\text{allreduce}}.$$

With ideal overlap between communication and interior computation:

$$T_{\text{iter}} \approx \max\left(T_{\text{interior}}, T_{\text{boundary}} + T_{\text{comm}}\right) + T_{\text{sync}} + T_{\text{allreduce}}.$$

The performance goal is to make:

$$T_{\text{boundary}} + T_{\text{comm}} \leq T_{\text{interior}},$$

so communication is mostly hidden.

Mathematical Reasoning

Stencil Dependency Radius

The Jacobi update uses a four-point stencil. The dependency radius is one grid cell:

$$u_{\text{new}}(i,j) = f\left(u(i-1,j), u(i+1,j), u(i,j-1), u(i,j+1)\right).$$

Because the dependency radius is one, each rank needs exactly one halo row from each vertical neighbor when using horizontal decomposition.

If the stencil radius were k, each rank would need k halo rows:

$$\text{halo depth} = k.$$

Communication Volume

For horizontal decomposition, each rank sends two rows per iteration: one to the top neighbor and one to the bottom neighbor. If each row has n_x elements and each element has size b bytes, the communication volume per rank per iteration is:

$$V_{\text{rank}} = 2n_x b.$$

For single-precision floats, $b = 4$, so:

$$V_{\text{rank}} = 8n_x \text{ bytes.}$$

The local computation volume per rank is approximately:

$$W_{\text{rank}} = n_x n_y^{\text{local}}$$

grid updates.

If:

$$n_y^{\text{local}} = \frac{n_y}{p},$$

then:

$$W_{\text{rank}} = \frac{n_x n_y}{p}.$$

Communication per rank does not shrink with p in this simple horizontal decomposition, but computation per rank does shrink with p. Therefore, the communication-to-computation ratio grows as p increases:

$$\frac{V_{\text{rank}}}{W_{\text{rank}}} \propto \frac{2n_x b}{n_x n_y / p} = \frac{2bp}{n_y}.$$

This explains why strong scaling becomes harder at larger GPU counts.

Parallel Efficiency

If a fraction f of the runtime is serial or communication-bound, Amdahl-style reasoning gives:

$$S_p = \frac{1}{f + \frac{1-f}{p}},$$

where S_p is the speedup on p GPUs. Efficiency is:

$$E_p = \frac{S_p}{p}.$$

As p grows, the term $\frac{1-f}{p}$ shrinks, and the nonparallel fraction f dominates. In multiGPU programs, f often includes:

- halo exchange latency,
- collective communication latency,
- host synchronization,
- kernel launch overhead,
- load imbalance,
- serial convergence checks.

Communication Latency Model

A common model for message transfer time is:

$$T_{\text{comm}}(m) = \alpha + \frac{m}{B},$$

where:

- α is startup latency,
- m is message size in bytes,
- B is effective bandwidth.

For small messages, latency dominates. For large messages, bandwidth dominates.

Halo rows may be small compared with large deep learning collectives, so latency can be very important.

All-Reduce for Convergence

The Jacobi solver computes a local residual contribution on each rank:

$$s_r = \sum_{(i,j)\in\Omega_r} \Delta_{i,j}^2,$$

where:

$$\Delta_{i,j} = u_{\text{new}}(i,j) - u(i,j).$$

Then an all-reduce sum computes:

$$s = \sum_{r=0}^{p-1} s_r.$$

The global residual is:

$$\text{residual} = \sqrt{s}.$$

The all-reduce is collective. All ranks participate, and all ranks receive the final result. This is different from a one-way halo exchange.

Code Walkthrough

MPI Program Skeleton

A minimal MPI program initializes MPI, queries rank and size, performs work, and finalizes MPI.

```
#include <mpi.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank = 0;
    int size = 0;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Each rank selects one GPU and computes one subdomain.

    MPI_Finalize();
    return 0;
}
```

Explanation:

- `MPI_Init` initializes the MPI runtime and establishes communication between processes.
- `MPI_Comm_rank` returns the current process rank.
- `MPI_Comm_size` returns the number of participating processes.
- The rank determines which part of the global domain the process owns.
- `MPI_Finalize` cleans up MPI resources.

In a GPU program, each rank usually calls `cudaSetDevice` to bind itself to a local GPU.

```
int local_rank = rank % num_gpus_per_node;
cudaSetDevice(local_rank);
```

This simple mapping works when each node has the same number of GPUs and ranks are packed by node.

Jacobi Kernel Structure

A representative CUDA kernel for one Jacobi update is:

```
__global__
void jacobi_kernel(
    const float* __restrict__ u,
    float* __restrict__ u_new,
    float* __restrict__ local_sum,
    int iy_start,
    int iy_end,
    int nx)
{
    int ix = blockIdx.x * blockDim.x + threadIdx.x;
    int iy = blockIdx.y * blockDim.y + threadIdx.y + iy_start;

    if (ix > 0 && ix < nx - 1 && iy < iy_end) {
        int idx = iy * nx + ix;

        float updated = 0.25f * (
            u[idx - 1] +
            u[idx + 1] +
            u[idx - nx] +
            u[idx + nx]);

        float diff = updated - u[idx];
        u_new[idx] = updated;

        atomicAdd(local_sum, diff * diff);
    }
}
```

Hardware explanation:

- Each thread computes one grid point.
- Consecutive threads in the x direction access consecutive memory locations.
- Reads of `u[idx - 1]` and `u[idx + 1]` are mostly coalesced.

- Reads of `u[idx - nx]` and `u[idx + nx]` are also coalesced across a warp if threads have consecutive `ix`.
- The kernel is memory-bandwidth-heavy because each update performs little arithmetic.
- The `atomicAdd` is simple but may become expensive. Production code often uses block reductions and then a final reduction.

MPI Halo Exchange

The lecture uses two send-receive operations to exchange top and bottom halo rows.

```
int top = (rank > 0) ? rank - 1 : size - 1;
int bottom = (rank < size - 1) ? rank + 1 : 0;

MPI_Sendrecv(
    &u[iy_start * nx], nx, MPI_FLOAT, top, 0,
    &u[iy_end * nx],   nx, MPI_FLOAT, bottom, 0,
    MPI_COMM_WORLD, MPI_STATUS_IGNORE);

MPI_Sendrecv(
    &u[(iy_end - 1) * nx], nx, MPI_FLOAT, bottom, 1,
    &u[(iy_start - 1) * nx], nx, MPI_FLOAT, top, 1,
    MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Explanation:

- The first call sends the first interior row upward and receives the bottom halo row.
- The second call sends the last interior row downward and receives the top halo row.
- `MPI_Sendrecv` avoids deadlock because send and receive are combined.
- The tags `0` and `1` distinguish the two logical message directions.
- If the MPI implementation is CUDA-aware, the pointers may point directly to GPU memory.

The memory layout is efficient because each row is contiguous. No packing kernel is needed.

One Iteration With MPI

A simplified structure of one Jacobi iteration is:

```
while (residual > tolerance && iter < max_iters) {
    cudaMemsetAsync(d_local_sum, 0, sizeof(float), compute_stream);
```

```
    jacobi_kernel<<<grid, block, 0, compute_stream>>>(
        u, u_new, d_local_sum, iy_start, iy_end, nx);

    cudaMemcpyAsync(
        &h_local_sum,
        d_local_sum,
        sizeof(float),
        cudaMemcpyDeviceToHost,
        compute_stream);

    cudaStreamSynchronize(compute_stream);

    exchange_halos_with_mpi(u_new, nx, iy_start, iy_end, rank, size);

    MPI_Allreduce(
        &h_local_sum,
        &h_global_sum,
        1,
        MPI_FLOAT,
        MPI_SUM,
        MPI_COMM_WORLD);

    residual = sqrt(h_global_sum);
    std::swap(u, u_new);
    ++iter;
}
```

The synchronization before MPI is needed because regular MPI calls are not CUDA stream operations. Without synchronization, MPI might read a boundary row before the kernel has finished writing it.

This is correct but exposes communication on the critical path.

Overlapping MPI Communication and Computation

The key optimization is to compute boundary rows first, start communication, and compute the interior while communication runs.

```
jacobi_kernel<<<top_grid, block, 0, push_top_stream>>>(
    u, u_new, d_local_sum_top, iy_start, iy_start + 1, nx);

jacobi_kernel<<<bottom_grid, block, 0, push_bottom_stream>>>(
    u, u_new, d_local_sum_bottom, iy_end - 1, iy_end, nx);

jacobi_kernel<<<inner_grid, block, 0, compute_stream>>>(
    u, u_new, d_local_sum_inner, iy_start + 1, iy_end - 1, nx);
```

```
cudaStreamSynchronize(push_top_stream);
send_top_receive_bottom_with_mpi(u_new, nx, iy_start, iy_end);

cudaStreamSynchronize(push_bottom_stream);
send_bottom_receive_top_with_mpi(u_new, nx, iy_start, iy_end);

cudaStreamSynchronize(compute_stream);
```

Explanation:

- The top boundary row is computed on `push_top_stream`.
- The bottom boundary row is computed on `push_bottom_stream`.
- The interior is computed on `compute_stream`.
- MPI communication begins as soon as the needed boundary row is ready.
- The larger interior kernel can run while MPI transfers boundary data.

This works because the interior rows do not depend on the newly received halo values until the next iteration.

NCCL Initialization Using MPI

NCCL requires all ranks to share a unique communicator ID. A common setup uses MPI to broadcast this ID.

```
#include <mpi.h>
#include <nccl.h>

ncclUniqueId id;

if (rank == 0) {
    ncclGetUniqueId(&id);
}

MPI_Bcast(&id, sizeof(id), MPI_BYTE, 0, MPI_COMM_WORLD);

ncclComm_t comm;
ncclCommInitRank(&comm, size, id, rank);

// NCCL communication can now be issued.

ncclCommDestroy(comm);
```

Explanation:

- Rank 0 creates a NCCL unique ID.

- MPI broadcasts the ID to all ranks.
- Each rank creates a NCCL communicator with its own rank and the global size.
- MPI and NCCL can coexist in the same program.

NCCL Point-to-Point Halo Exchange

NCCL supports point-to-point send and receive operations. Unlike `MPI_Sendrecv`, NCCL does not use message tags, so operation ordering must match.

```
ncclGroupStart();

ncclRecv(
    &u_new[iy_end * nx],
    nx,
    ncclFloat,
    bottom,
    comm,
    compute_stream);

ncclSend(
    &u_new[iy_start * nx],
    nx,
    ncclFloat,
    top,
    comm,
    compute_stream);

ncclRecv(
    &u_new[(iy_start - 1) * nx],
    nx,
    ncclFloat,
    top,
    comm,
    compute_stream);

ncclSend(
    &u_new[(iy_end - 1) * nx],
    nx,
    ncclFloat,
    bottom,
    comm,
    compute_stream);
```

```
ncclGroupEnd();
```

Explanation:

- `ncclGroupStart` and `ncclGroupEnd` group operations together.
- Grouping can reduce launch overhead and give NCCL more optimization freedom.
- NCCL operations are enqueued on a CUDA stream.
- If the Jacobi kernel is already in the same stream, the communication is naturally ordered after it.
- No explicit `cudaStreamSynchronize` is needed between the kernel and NCCL operation when they use the same stream.

The key distinction from MPI is:

$$\text{MPI call} \notin \text{CUDA stream}$$

while:

$$\text{NCCL call} \in \text{CUDA stream order.}$$

NCCL Overlap With Stream Priorities

To overlap NCCL communication with computation, boundary kernels and NCCL communication are placed on a high-priority stream, while the interior compute kernel runs on a lower-priority stream.

```
int least_priority = 0;
int greatest_priority = 0;

cudaDeviceGetStreamPriorityRange(
    &least_priority,
    &greatest_priority);

cudaStream_t compute_stream;
cudaStream_t push_stream;

cudaStreamCreateWithPriority(
    &compute_stream,
    cudaStreamNonBlocking,
    least_priority);

cudaStreamCreateWithPriority(
    &push_stream,
    cudaStreamNonBlocking,
```

```
    greatest_priority);

jacobi_kernel<<<top_grid, block, 0, push_stream>>>(
    u, u_new, d_sum_top, iy_start, iy_start + 1, nx);

jacobi_kernel<<<bottom_grid, block, 0, push_stream>>>(
    u, u_new, d_sum_bottom, iy_end - 1, iy_end, nx);

jacobi_kernel<<<inner_grid, block, 0, compute_stream>>>(
    u, u_new, d_sum_inner, iy_start + 1, iy_end - 1, nx);

ncclGroupStart();
// Enqueue NCCL sends and receives on push_stream.
ncclGroupEnd();
```

Explanation:

- Stream priority helps ensure boundary work is not delayed behind the large interior kernel.
- The high-priority stream computes boundary rows first.
- The NCCL communication kernel then runs early enough to overlap with interior computation.
- Without priority, the large interior kernel might monopolize the GPU, delaying boundary communication until too late.

NVSHMEM Initialization With MPI

NVSHMEM can be initialized using an existing MPI communicator.

```
#include <mpi.h>
#include <nvshmem.h>
#include <nvshmemx.h>

MPI_Comm mpi_comm = MPI_COMM_WORLD;

nvshmemx_init_attr_t attr;
attr.mpi_comm = &mpi_comm;

nvshmemx_init_attr(
    NVSHMEMX_INIT_WITH_MPI_COMM,
    &attr);

int mype = nvshmem_my_pe();
int npes = nvshmem_n_pes();

// Use NVSHMEM allocations and communication.
```

```
nvshmem_finalize();
```

Explanation:

- MPI launches and connects the processes.
- NVSHMEM uses the MPI communicator to establish its PE group.
- `nvshmem_my_pe` is analogous to MPI rank.
- `nvshmem_n_pes` is analogous to MPI size.

NVSHMEM Symmetric Allocation

Communication buffers must be allocated in the symmetric heap.

```
float* u = static_cast<float*>(
    nvshmem_malloc(num_elements * sizeof(float)));

float* u_new = static_cast<float*>(
    nvshmem_malloc(num_elements * sizeof(float)));

int* sync = static_cast<int*>(
    nvshmem_malloc(sizeof(int)));

cudaMemset(u, 0, num_elements * sizeof(float));
cudaMemset(u_new, 0, num_elements * sizeof(float));
cudaMemset(sync, 0, sizeof(int));
```

Explanation:

- `nvshmem_malloc` creates symmetric memory visible to other PEs.
- The same symbolic allocation exists on each PE.
- One PE can issue one-sided writes to another PE's symmetric allocation.
- Private GPU memory can still be allocated with `cudaMalloc`.

NVSHMEM In-Kernel Single-Element Put

NVSHMEM allows communication from inside a CUDA kernel.

```
__global__
void jacobi_nvshmem_kernel(
    float* u,
    float* u_new,
    int nx,
    int iy_start,
    int iy_end,
    int top_pe,
```

```
    int bottom_pe)
{
    int ix = blockIdx.x * blockDim.x + threadIdx.x;
    int iy = blockIdx.y * blockDim.y + threadIdx.y + iy_start;

    if (ix > 0 && ix < nx - 1 && iy < iy_end) {
        int idx = iy * nx + ix;

        float updated = 0.25f * (
            u[idx - 1] +
            u[idx + 1] +
            u[idx - nx] +
            u[idx + nx]);

        u_new[idx] = updated;

        if (iy == iy_start) {
            nvshmem_float_p(
                &u_new[(iy_end) * nx + ix],
                updated,
                top_pe);
        }

        if (iy == iy_end - 1) {
            nvshmem_float_p(
                &u_new[(iy_start - 1) * nx + ix],
                updated,
                bottom_pe);
        }
    }
}
```

Explanation:

- Each thread computes one grid point.
- Boundary-row threads also send their computed value to a neighboring PE.
- `nvshmem_float_p` performs a one-sided put of one float.
- The remote PE does not post a matching receive.
- This can be efficient on NVLink-like fabrics where remote stores map naturally to hardware operations.

NVSHMEM Block-Level Put

For larger messages, especially over InfiniBand, it is often better to communicate cooperatively at block granularity.

```
__global__
void boundary_put_kernel(
    float* u_new,
    int nx,
    int iy_start,
    int iy_end,
    int top_pe,
    int bottom_pe)
{
    if (blockIdx.y == 0) {
        nvshmemx_float_put_block(
            &u_new[iy_end * nx],
            &u_new[iy_start * nx],
            nx,
            bottom_pe);
    }

    if (blockIdx.y == gridDim.y - 1) {
        nvshmemx_float_put_block(
            &u_new[(iy_start - 1) * nx],
            &u_new[(iy_end - 1) * nx],
            nx,
            top_pe);
    }
}
```

Explanation:

- Block-level put lets a block cooperate on a larger transfer.
- Larger transfers better match network interfaces that prefer bigger messages.
- This is less fine-grained than per-thread single-float puts.
- It can improve bandwidth utilization.

Fully Persistent NVSHMEM Kernel

A more advanced pattern moves the iteration loop into one long-running kernel. This reduces repeated kernel launch overhead and enables fine-grained communication inside the kernel.

```
__global__
void persistent_jacobi_kernel(
```

```
    float* u,
    float* u_new,
    int* sync,
    int nx,
    int iy_start,
    int iy_end,
    int top_pe,
    int bottom_pe,
    int max_iters)
{
    for (int iter = 0; iter < max_iters; ++iter) {
        int ix = blockIdx.x * blockDim.x + threadIdx.x;
        int iy = blockIdx.y * blockDim.y + threadIdx.y + iy_start;

        if (ix > 0 && ix < nx - 1 && iy < iy_end) {
            int idx = iy * nx + ix;

            float updated = 0.25f * (
                u[idx - 1] +
                u[idx + 1] +
                u[idx - nx] +
                u[idx + nx]);

            u_new[idx] = updated;
        }

        __syncthreads();

        if (blockIdx.y == 0) {
            nvshmemx_float_put_block(
                &u_new[iy_end * nx],
                &u_new[iy_start * nx],
                nx,
                bottom_pe);
        }

        if (blockIdx.y == gridDim.y - 1) {
            nvshmemx_float_put_block(
                &u_new[(iy_start - 1) * nx],
                &u_new[(iy_end - 1) * nx],
                nx,
                top_pe);
        }

        nvshmem_quiet();
```

```
        if (threadIdx.x == 0 && threadIdx.y == 0) {
            if (blockIdx.y == 0) {
                nvshmem_int_atomic_inc(sync, top_pe);
            }

            if (blockIdx.y == gridDim.y - 1) {
                nvshmem_int_atomic_inc(sync, bottom_pe);
            }
        }

        int target = 2 * gridDim.x * (iter + 1);
        nvshmem_int_wait_until(sync, NVSHMEM_CMP_GE, target);

        float* tmp = u;
        u = u_new;
        u_new = tmp;
    }
}
```

Explanation:

- The outer iteration loop is inside the CUDA kernel.
- Boundary data is communicated from inside the kernel.
- `nvshmem_quiet` ensures previous NVSHMEM operations are complete.
- Atomic increments notify neighboring PEs that boundary communication has completed.
- `nvshmem_int_wait_until` implements a device-side synchronization condition.
- The target count grows each iteration because the sync variable is not reset.

This pattern is powerful but requires careful reasoning about scheduling, synchronization, and whether all communicating blocks can be resident when needed.

Performance Intuition

Why CUDA-Aware MPI Helps

CUDA-aware MPI lets the programmer pass device pointers directly to MPI. This avoids manual staging and allows the MPI implementation to choose the best path.

Best case:

$$\text{GPU} \to \text{NIC} \to \text{network} \to \text{NIC} \to \text{GPU}.$$

Manual staging case:

$$\text{GPU} \to \text{CPU} \to \text{NIC} \to \text{network} \to \text{NIC} \to \text{CPU} \to \text{GPU}.$$

The staged path adds bandwidth pressure, latency, synchronization, and CPU involvement.

Why Overlap Matters

Suppose one iteration has:

$$T_{\text{interior}} = 100\ \mu s, \quad T_{\text{boundary}} = 5\ \mu s, \quad T_{\text{comm}} = 20\ \mu s.$$

Without overlap:

$$T_{\text{iter}} = 100 + 5 + 20 = 125\ \mu s.$$

With ideal overlap:

$$T_{\text{iter}} = \max(100, 5 + 20) = 100\ \mu s.$$

The communication is hidden. But if strong scaling reduces the interior time to:

$$T_{\text{interior}} = 10\ \mu s,$$

then:

$$T_{\text{iter}} = \max(10, 25) = 25\ \mu s.$$

Now communication dominates again. This is why strong scaling becomes harder at high GPU counts.

Why NCCL Can Remove Synchronization

MPI does not naturally participate in CUDA stream ordering. Therefore, the CPU often needs to synchronize before MPI reads data produced by a kernel.

NCCL operations are enqueued on CUDA streams. If a kernel writes a boundary row and a NCCL send is enqueued after it in the same stream, CUDA stream ordering guarantees correctness:

$$\text{boundary kernel} \prec \text{NCCL send}.$$

This reduces host-side synchronization.

Why Stream Priority Matters

A large interior kernel may consume most GPU resources. If boundary kernels and communication are placed on normal streams, they may not run early enough to overlap. A high-priority stream gives the boundary work a better chance to execute promptly.

The desired timeline is:

$$\text{boundary compute} \rightarrow \text{communication}$$

running concurrently with:

$$\text{interior compute}.$$

Stream priority helps create this timeline.

Why NVSHMEM Can Be Faster

NVSHMEM can communicate from within a kernel. This enables:

- fewer kernel launches,
- kernel fusion,
- device-side communication,
- one-sided communication,
- lower host involvement,
- finer-grained overlap.

This is especially valuable when strong scaling makes each local subproblem small. At large GPU counts, launch overhead and synchronization latency can dominate, so avoiding them becomes important.

Why NVSHMEM May Not Always Win

NVSHMEM is not automatically faster in every application. A simple Jacobi solver has only one main kernel, so there may be little opportunity for fusion. If the communication is too fine-grained or the network path prefers larger messages, single-element puts may underutilize bandwidth.

NVSHMEM is most compelling when the algorithm benefits from in-kernel communication or when multiple separate kernels and communication phases can be fused.

Common Misconceptions

Misconception: More GPUs Always Means Faster Runtime

More GPUs increase compute capacity, but they also increase communication and synchronization. In strong scaling, local work shrinks as GPU count grows, so overhead becomes more visible.

Correct mental model:

$$\text{speedup} = \frac{\text{removed compute time}}{\text{added communication and synchronization cost}}.$$

Misconception: MPI Is Only for CPUs

Modern CUDA-aware MPI implementations can send and receive GPU buffers directly. When GPU Direct RDMA or peer-to-peer is available, the transfer can avoid host staging.

Misconception: NCCL Is Only for All-Reduce

NCCL is historically known for collectives, especially all-reduce in deep learning. But NCCL also supports point-to-point send and receive. Its stream-ordered interface is useful beyond neural network training.

Misconception: NVSHMEM Replaces MPI and NCCL

NVSHMEM is not a universal replacement. It provides a different model. MPI, NCCL, and NVSHMEM can interoperate. A program may use MPI for launch and CPU-side control, NCCL for collectives, and NVSHMEM for in-kernel communication.

Misconception: One Process Per GPU Is a Hard Universal Rule

One process per GPU is a strong convention and often the best-performing model, but it is not the only possible model. A single process can manage multiple GPUs, but the program becomes more complex because it must manage multiple contexts, devices, streams, and memory regions.

Misconception: Communication Bandwidth Is the Only Metric

Latency is often more important for small halo exchanges. A transfer model includes both startup latency and bandwidth:

$$T_{\text{comm}} = \alpha + \frac{m}{B}.$$

For small m, the α term dominates.

Practical Takeaways

Choose the Communication Model Based on the Algorithm

Use CUDA-aware MPI when:

- the code already uses MPI,
- the algorithm has natural message-passing structure,
- CPU and GPU communication coexist,
- you want a mature and portable distributed programming model.

Use NCCL when:

- communication is between GPU buffers,
- collectives such as all-reduce are important,
- stream ordering matters,
- you want to reduce host synchronization.

Use NVSHMEM when:

- one-sided communication simplifies the algorithm,
- device-side communication is valuable,
- kernel fusion can remove launch overhead,
- communication must happen inside CUDA kernels.

Always Understand the Critical Path

Before optimizing, ask:

- Which operations must happen before communication can start?
- Which communication operations block progress?
- Can boundary work be separated from interior work?

- Can communication be placed on a CUDA stream?
- Can synchronization be delayed or avoided?
- Is the application bandwidth-bound, latency-bound, or compute-bound?

Profile With Nsight Systems

Nsight Systems helps reveal:

- CUDA kernel timelines,
- CUDA memory copies,
- MPI calls,
- NCCL kernels,
- overlap between compute and communication,
- PCIe or NVLink traffic,
- CPU synchronization gaps.

A good multiGPU profile should show communication hidden behind computation whenever possible.

Think in Terms of Local Work Versus Boundary Surface

For stencil codes, the communication cost is proportional to boundary size, while computation is proportional to volume.

For a two-dimensional tile:

$$\text{compute} \propto n_x n_y,$$

while boundary exchange is approximately:

$$\text{communication} \propto 2n_x + 2n_y.$$

For a three-dimensional block:

$$\text{compute} \propto n_x n_y n_z,$$

while boundary exchange is approximately:

$$\text{communication} \propto 2(n_x n_y + n_x n_z + n_y n_z).$$

Large subdomains have better compute-to-communication ratios. Strong scaling shrinks subdomains and worsens that ratio.

Project or Experiment Ideas

Experiment 1: Single-GPU Jacobi Baseline

Implement the Jacobi solver on one GPU. Measure:

- runtime per iteration,
- effective memory bandwidth,
- achieved occupancy,
- global memory load efficiency,
- effect of block size.

Compute arithmetic intensity:

$$I \approx 0.2 \text{ FLOPs/byte}.$$

Check whether measured performance matches the expectation of a memory-bound kernel.

Experiment 2: MPI Halo Exchange

Extend the solver to multiple ranks with CUDA-aware MPI. Compare:

- GPU Direct RDMA enabled,
- CUDA-aware MPI with staging,
- manual device-to-host staging.

Measure:

$$T_{\text{comm}}(m) = \alpha + \frac{m}{B}$$

for different row sizes.

Experiment 3: Overlap Communication With Computation

Split the Jacobi update into top boundary, bottom boundary, and interior kernels. Use separate streams and compare:

- no overlap,
- MPI overlap,
- NCCL stream-ordered overlap,
- NCCL overlap with stream priority.

Plot parallel efficiency:

$$E_p = \frac{T_1}{pT_p}.$$

Experiment 4: NCCL Point-to-Point Versus MPI

Implement the same halo exchange with NCCL point-to-point and MPI. Compare:

- code complexity,
- required synchronization,
- timeline overlap,
- scaling from 2 to 8 GPUs,
- sensitivity to message size.

Experiment 5: NVSHMEM In-Kernel Communication

Implement boundary exchange using NVSHMEM from inside the kernel. Compare:

- single-element put,
- block-level put,
- host-side loop with one kernel per iteration,
- persistent kernel with device-side synchronization.

Measure whether the gain comes from:

- reduced launch overhead,
- better overlap,
- reduced host synchronization,
- fewer intermediate kernels.

Experiment 6: Strong Scaling Study

Fix the global domain size and run on:

$$1, 2, 4, 8, 16$$

GPUs if available. Plot:

$$T_p, \quad S_p = \frac{T_1}{T_p}, \quad E_p = \frac{S_p}{p}.$$

Interpret where scaling breaks down and whether the bottleneck is communication, synchronization, or insufficient local work.

Final Mental Model

MultiGPU programming is single-GPU programming plus distributed systems reasoning. A single GPU kernel optimizes memory access, occupancy, and arithmetic intensity. A multiGPU application additionally optimizes decomposition, communication paths, synchronization, launch overhead, and critical-path overlap.

For the Jacobi solver, the core idea is simple: each GPU owns a stripe of the domain, computes its interior, and exchanges halo rows with neighbors. The performance challenge is not the stencil formula. The performance challenge is making the required communication happen without exposing its latency.

MPI gives a mature process-level message-passing model. CUDA-aware MPI can send GPU buffers directly, especially with GPU Direct. NCCL gives GPU-buffer communication with CUDA stream ordering, making it easier to remove unnecessary synchronization. NVSHMEM gives a one-sided partitioned global address space model and enables communication directly from CUDA kernels.

The most important engineering rule is:

> Do not merely ask whether communication is fast. Ask whether communication is on the critical path.

A good multiGPU implementation decomposes the problem so that each GPU has enough local work, computes boundary data early, starts communication as soon as possible, overlaps communication with interior computation, uses GPU Direct paths when available, and chooses the communication library whose execution model matches the algorithm.